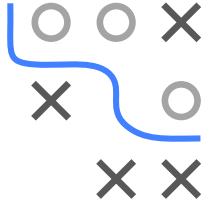
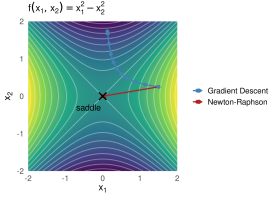
Optimization in Machine Learning



Second order methods
Limitations of Newton-Raphson

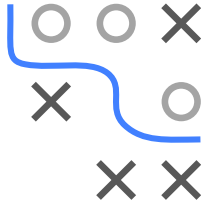


Learning goals

- Singular and indefinite Hessian
- Non-descent directions
- Saddle points
- Computational cost

Slides based on
▸ Aggarwal 2020 (Ch. 5.6)

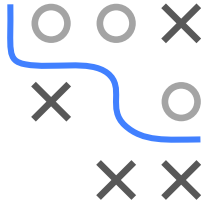
LIMITATION 1: SINGULAR AND INDEFINITE HESSIAN



- **Problem**: Newton-Raphson is designed for *convex quadratic f* with pd Hessian, but $\mathbf{H} = \nabla^2 f(\boldsymbol{x}^{[t]})$ may be
  - **singular**: $\nabla^2 f(\boldsymbol{x}^{[t]})\mathbf{d}^{[t]} = -\nabla f(\boldsymbol{x}^{[t]})$ has no unique solution $\Rightarrow$ step undefined
  - **near-singular**: step length blows up as $\lambda_{\min} \to 0$, and errors already present in $\mathbf{H}$ and ∇f (minibatch noise, finite-difference approximations, floating-point roundoff) are amplified by $\kappa(\mathbf{H}) = |\lambda_{\max}|/|\lambda_{\min}|$
  - **indefinite**: $\mathbf{d}^{[t]}$ is not a descent direction (Limitation 2), saddle points (Limitation 3)

- **When does (near-)singularity happen in ML?** Rank-deficient design, collinear features, $d > n$, or degenerate optima

EXAMPLE: SINGULAR HESSIAN IN THE L_2-SVM



- Unregularized L_2-SVM (squared hinge loss) with $\boldsymbol{\theta} \in \mathbb{R}^d$:

$$\mathcal{R}_{\text{emp}}(\boldsymbol{\theta}) = \sum_{i=1}^{n} \max\{1 - y^{(i)}\boldsymbol{\theta}^T\boldsymbol{x}^{(i)},\ 0\}^2$$

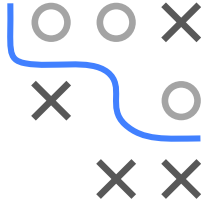
- Only margin violators $\mathcal{M} = \{i : y^{(i)}\boldsymbol{\theta}^T\boldsymbol{x}^{(i)} < 1\}$ contribute to $\mathbf{H} \in \mathbb{R}^{d \times d}$:

$$\mathbf{H} = 2 \sum_{i \in \mathcal{M}} \boldsymbol{x}^{(i)}(\boldsymbol{x}^{(i)})^T$$

- Each $\boldsymbol{x}^{(i)}(\boldsymbol{x}^{(i)})^T$ has **rank 1** $\Rightarrow \text{rank}(\mathbf{H}) \leq |\mathcal{M}|$, so we need at least d margin violators for $\mathbf{H}$ to be invertible

- **But**: the better our fit, the fewer violators $\Rightarrow$ $\mathbf{H}$ becomes singular exactly where we want to converge

REMEDY: REGULARIZED NEWTON STEP

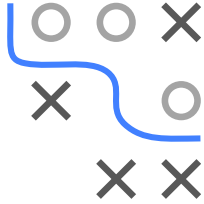


- Replace $\mathbf{H}$ by $\mathbf{H} + \lambda \boldsymbol{I}$ with $\lambda > 0$: all eigenvalues are shifted by λ

$$(\mathbf{H} + \lambda \boldsymbol{I})\mathbf{d}^{[t]} = -\nabla f(\boldsymbol{x}^{[t]})$$

- Choosing λ slightly larger than $|\lambda_{\min}|$ makes $\mathbf{H} + \lambda \boldsymbol{I}$ pd $\Rightarrow$ invertible and descent direction
- Alternative: use pseudoinverse $\mathbf{H}^{+}$ instead of $\mathbf{H}^{-1}$
- **Caveat**: regularization does not cure ill-conditioning – for nearly singular $\mathbf{H}$ the step stays sensitive to errors in $\mathbf{H}$ and ∇f

LIMITATION 2: NON-DESCENT DIRECTIONS

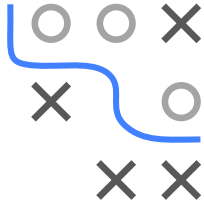


- **Problem**: In general, $\mathbf{d}^{[t]}$ is not a descent direction
- **But**: If Hessian is positive definite, $\mathbf{d}^{[t]}$ is descent direction:

$$\nabla f(\boldsymbol{x}^{[t]})^T \mathbf{d}^{[t]} = -\nabla f(\boldsymbol{x}^{[t]})^T (\nabla^2 f(\boldsymbol{x}^{[t]}))^{-1} \nabla f(\boldsymbol{x}^{[t]}) < 0$$

- Near minimum, Hessian is positive definite
- For initial steps, Hessian is often not positive definite and Newton-Raphson may give non-descending update directions

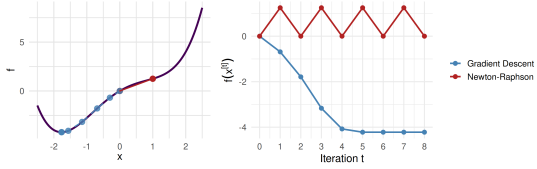
EXAMPLE: NON-DESCENT DIRECTIONS



- Consider $f(x) = \frac{x^4}{4} - x^2 + 2x$ with $f'(x) = x^3 - 2x + 2$, $f''(x) = 3x^2 - 2$
- Starting at $x^{[0]} = 0$: $f''(0) = -2 < 0$ (**H** not pd) $\Rightarrow$ Newton-Raphson performs **ascent** step:

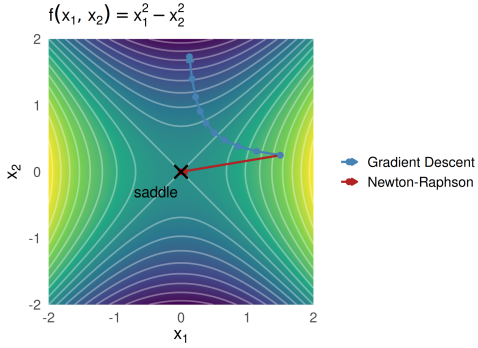
$$x^{[1]} = x^{[0]} - \frac{f'(x^{[0]})}{f''(x^{[0]})} = 0 - \frac{2}{-2} = 1, \qquad f(x^{[1]}) = 1.25 > 0 = f(x^{[0]})$$

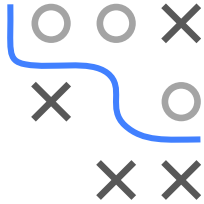
- From $x^{[1]} = 1$: $f''(1) = 1 > 0$, but the step leads right back to $x^{[2]} = 0$ $\Rightarrow$ Newton-Raphson cycles forever between 0 and 1
- **GD** from the same $x^{[0]} = 0$ only uses the neg. gradient $-f'(x^{[0]}) = -2$ (descent direction) and converges for appropriate step size α



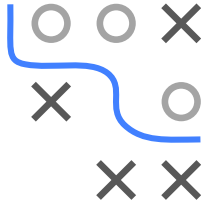
LIMITATION 3: SADDLE POINTS

- **Problem**: Newton-Raphson solves $\nabla f(\boldsymbol{x}) = \mathbf{0} \Rightarrow$ attracted to **any** critical point (minima, maxima, saddles)
- **Example**: $f(x_1, x_2) = x_1^2 - x_2^2$ with $\mathbf{H} = \begin{pmatrix} 2 & 0 \\ 0 & -2 \end{pmatrix}$ indefinite
  - Since f is quadratic, a single Newton step lands *exactly* on the saddle $(0, 0)$, from any starting point
  - GD is repelled along the negative curvature direction x_2 and escapes (as long as $x_2^{[0]} \neq 0$)

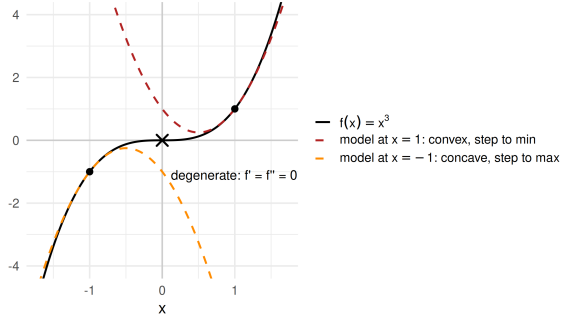




SADDLE POINTS: APPROXIMATION FLIPS

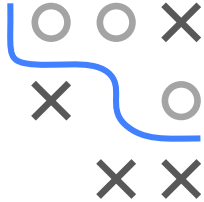


- Near an inflection point, the quadratic approximation flips, e.g., for $f(x) = x^3$:



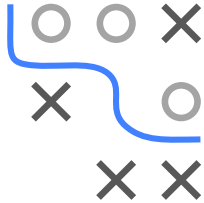
- At $x = 0$: $f' = f'' = 0 \Rightarrow$ update $0/0$ undefined (**degenerate critical point**)

SADDLE POINTS: WHY THIS MATTERS



- In high-dimensional non-convex problems (e.g., DNNs), saddle points vastly outnumber true optima
- **GD can escape** saddle points: it is not attracted by them but may still get stuck in flat regions
- **Newton-Raphson cannot escape** saddle points: it is attracted *indiscriminately* to all critical points
- Second order is not uniformly better: it pays off for complex curvature, but not for landscapes riddled with saddles
- Practitioners often prefer first order methods with adaptive schemes (e.g., Adam), which pick up some curvature information implicitly and at much lower cost (more on this in the next chapter)

LIMITATION 4: COMPUTATIONAL COST



- **Problem**: The Newton-Raphson update can be **computationally expensive**, as $\mathbf{d}^{[t]}$ is solution of the linear system

$$\nabla^2 f(\boldsymbol{x}^{[t]}) \mathbf{d}^{[t]} = -\nabla f(\boldsymbol{x}^{[t]})$$

- For $\boldsymbol{x} \in \mathbb{R}^d$, the Hessian has d^2 entries: $\mathcal{O}(d^2)$ memory, and $\mathcal{O}(d^3)$ per iteration to solve the linear system (compared to $\mathcal{O}(d)$ memory and no linear solve for GD)
- For large d (e.g., deep neural networks with $d \sim 10^9$ parameters), this is prohibitive: storing a dense $d \times d$ Hessian alone is infeasible
- **In ML**: for ERM with n observations, already forming $\mathbf{H}$ costs another $\mathcal{O}(nd^2)$ (compared to $\mathcal{O}(nd)$ for $\nabla \mathcal{R}_{\text{emp}}(\boldsymbol{\theta})$)
- Newton-Raphson needs far fewer iterations than GD, but for large d the cost *per* iteration dominates
- **Aim:** Find quasi-second order methods not relying on exact $\mathbf{H}$
  - Quasi-Newton method
  - Gauss-Newton algorithm (for least squares)